In-Between Co-pilot
Frontend Worklog

Project contribution record

25 July 2026

Contents

1 Introduction

This worklog records frontend and related backend contributions to the In-Between Co-pilot. The product assists anime artists by generating in-between frames from key drawings and presenting QA results for review. The detailed contribution record is consolidated in the Progress section and is based on the evidence in `frontend/docs/worklog.md`.

2 Design

The artist workspace follows a chat-first interaction model with a review board for detailed inspection. It combines a composer, media import, an onboarding state, navigation, and review affordances. The interface uses the existing Tailwind and shadcn/ui design system.

3 System architecture

The frontend is a static Next.js application served with the FastAPI service in production. In development, relative API requests are rewritten to the configured service target. The browser submits media to the service and consumes its decision log through server-sent events read with `fetch()` and a `ReadableStream`.

The frontend provides the artist interaction surface, while the backend runs the generation and QA pipeline, returns per-pair and final events, and exposes protected session resources.

4 Frontend implementation

The frontend uses Next.js App Router, React, TypeScript, Tailwind CSS, and shadcn/ui. The public landing page is statically renderable, while the protected `/copilot` route loads the browser-only workspace client-side.

The workspace is designed for media submission, live QA feedback, artist review, and session-oriented interaction. Specific delivered frontend work is listed in the Progress section.

5 Backend contribution

The FastAPI backend provides the client authentication boundary, protected session resources, and the session-history surface consumed by the frontend. It enforces authenticated access before serving protected user resources.

5.1 Authentication

Authentication pages support sign-in, sign-up with email confirmation, and password reset through the AWS Amplify SDK and Cognito. Cognito and Google authentication were configured for the application.

The current security boundary is a one-time Cognito ID-token handoff to `/auth/session`. FastAPI validates the token and sets an HttpOnly application cookie. Subsequent protected API and SSE calls use same-origin cookie credentials; browser clients do not directly access DynamoDB or S3.

Figure 1 shows the direct Cognito registration path. The browser receives the Cognito ID token only for the one-time application-session bootstrap. The application session itself is the cookie set by FastAPI; the final protected requests therefore carry that cookie rather than a browser-managed bearer token.

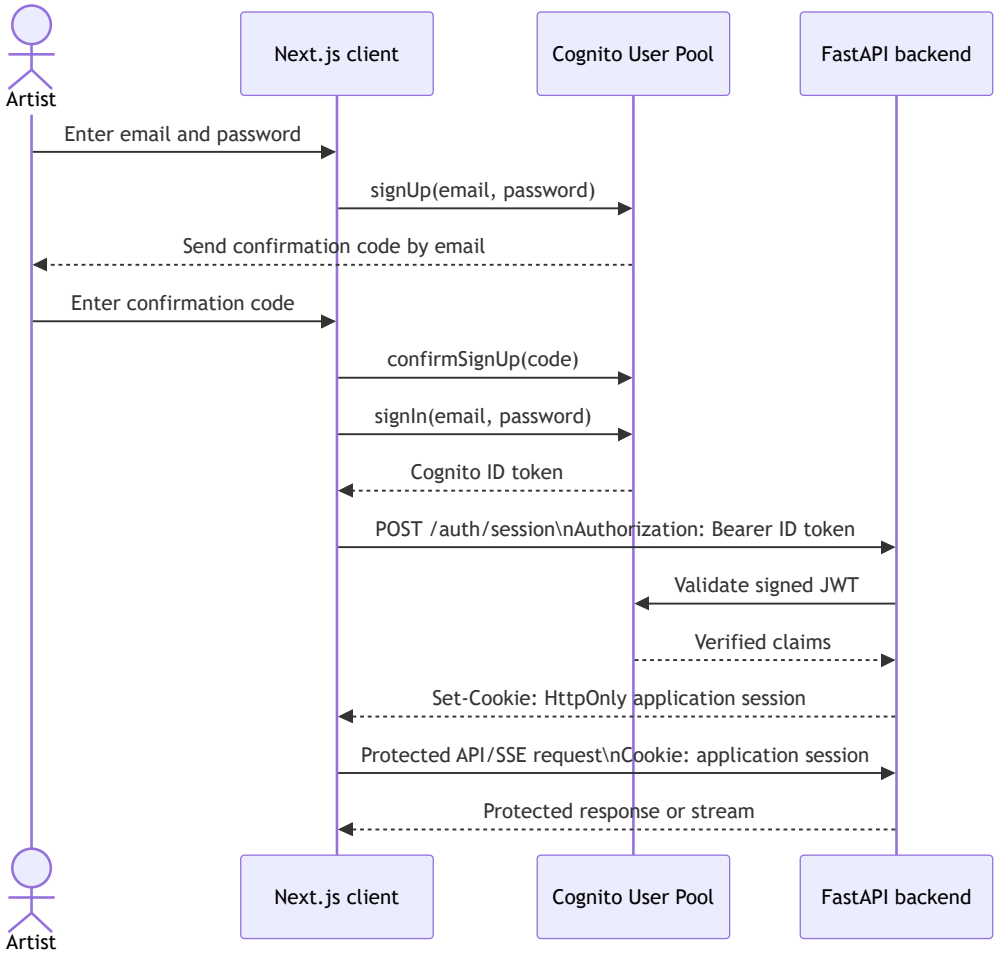

Figure 1: Direct Cognito sign-up, confirmation, and cookie-session bootstrap.

Figure 2 shows the corresponding Google OAuth route. Cognito's hosted UI mediates the provider redirect and code exchange. After Amplify obtains the Cognito ID token, the same FastAPI cookie-session bootstrap is used, so all subsequent protected client requests use the backend-issued cookie.

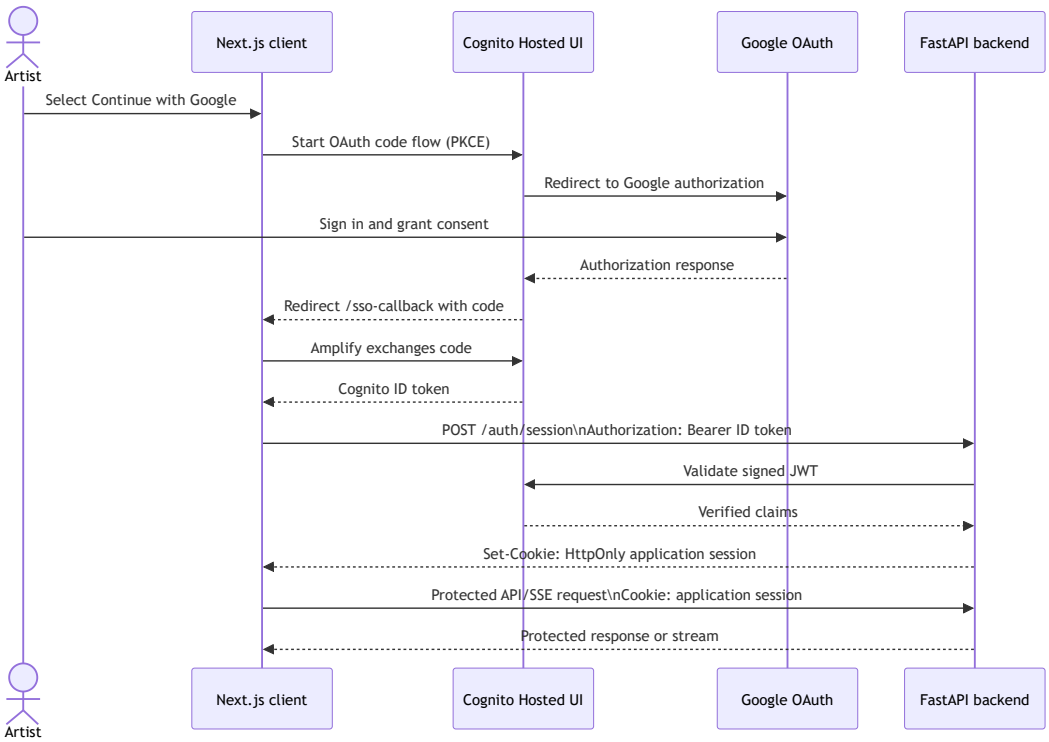

Figure 2: Google OAuth through Cognito, followed by the same backend cookie-session boundary.

5.2 Storage and session history

Session metadata is owned by the authenticated user and supports sidebar history. Completed workspaces can be stored as a snapshot for later use. The detailed storage contributions and their recorded verification are listed in the Progress section.

6 Progress

This section is the complete contribution record.

1. **Initial interface and integration.** Implemented an initial main-region chatbox interface and sidebar without application logic. Merged it with the teammate's existing basic file-picker/composer-like component, which had no text input, and the preview-board layout. After the merge, took responsibility for the frontend and part of the backend work.
2. **Chat Composer refinement.** Refined the Chat Composer layout and fixed media-selection behaviour so media not belonging to the selected input mode is not imported.
3. **Interface copy correction.** Corrected typographical issues where Japanese and English text were mixed.
4. **Welcome state.** Replaced the main-region multiplane animation with a simpler Welcome Message component that provides quick media-import actions.
5. **Sidebar.** Added the sidebar as part of the artist workspace navigation.
6. **Landing and authentication pages.** Developed all landing-page sections and the authentication-related pages.
7. **Amplify authentication logic.** Implemented authentication with the AWS Amplify SDK, supporting sign-in, sign-up with email confirmation, and password reset.
8. **Cognito and Google authentication configuration.** Configured Cognito and Google Authentication for the authentication flow.
9. **Backend cookie-authentication routes.** Implemented `/auth/me`, `/auth/session`, and `/auth/signout` for client requests. Enforced client-cookie verification on existing routes so unauthenticated clients cannot request protected S3 or DynamoDB resources.
10. **Owner-scoped session metadata.** Modified DynamoDB `session_metadata` to include `user_sub`, added a global secondary index for faster queries, and configured the title attribute for sidebar display.
11. **Completed-workspace storage.** Added a mechanism that stores `workspace.json` in the S3 bucket after a session completes.
12. **History-session retrieval.** Implemented routes that retrieve the user's history sessions and allow the browser to display retrieved `session_metadata` in the console.
13. **Hosting synchronisation.** Synchronised the backend codebase to the teammate's hosting computer.
14. **Recorded integration test.** Tested that a logged-in user can view their own sessions, and that creating a new session creates a new DynamoDB `session_metadata` entry that is loaded in the sidebar.

15. **Worklog authentication-diagram maintenance.** Replaced the authentication section's Cognito figure reference with the supplied `cognito-auth-sequence.pdf` figure and retained the supplied `google-oauth-cookie.pdf` figure.

16. **Worklog figure-placement correction.** Forced both authentication figures to remain in the Authentication section, preventing LaTeX float placement from moving the Google OAuth diagram after the Progress heading.

17. **Mermaid authentication-diagram rendering.** Re-rendered both authentication diagrams directly from their Mermaid sources with Mermaid CLI and its `pdfFit` option, replacing the supplied letter-sized PDFs with tightly fitted outputs.

18. **Worklog backend-section structure.** Moved Authentication and Storage and session history under Backend contribution as subsections, while preserving both Mermaid-generated authentication figures.

7 Conclusion

The contribution establishes an artist-oriented frontend backed by authenticated session and storage services. It includes the workspace experience, public and authentication entry points, cookie-protected backend integration, and owner-scoped session history. The complete, evidence-based record is provided in the Progress section.